
There Was Never a Bottleneck in Concept Bottleneck Models

Antonio Almudévar
ViVoLab, I3A
University of Zaragoza
almudevar@unizar.es

José Miguel Hernández-Lobato
Machine Learning Group
University of Cambridge
jmh233@cam.ac.uk

Alfonso Ortega
ViVoLab, I3A
University of Zaragoza
ortega@unizar.es

Abstract

Deep learning representations are often difficult to interpret, which can hinder their deployment in sensitive applications. Concept Bottleneck Models (CBMs) have emerged as a promising approach to mitigate this issue by learning representations that support target task performance while ensuring that each component predicts a concrete concept from a predefined set. In this work, we argue that CBMs do not impose a true bottleneck: the fact that a component can predict a concept does not guarantee that it encodes only information about that concept. This shortcoming raises concerns regarding interpretability and the validity of intervention procedures. To overcome this limitation, we propose Minimal Concept Bottleneck Models (MCBMs), which incorporate an Information Bottleneck (IB) objective to constrain each representation component to retain only the information relevant to its corresponding concept. This IB is implemented via a variational regularization term added to the training loss. As a result, MCBMs support concept-level interventions with theoretical guarantees, remain consistent with Bayesian principles, and offer greater flexibility in key design choices.

1 Introduction

Deep learning (DL) models have become the de facto standard for solving a wide range of tasks, primarily due to their ability to uncover complex patterns in data. However, these patterns often go beyond human comprehension, leading to DL models being perceived as black boxes—their predictions may be accurate, but they are typically difficult to interpret. Despite this, interpretability remains a crucial requirement in several domains, including but not limited to healthcare [1–3], finance [4, 5], and autonomous driving [6, 7].

Most DL models operate by learning data representations—compressed versions of the input that retain essential information for tackling specific tasks of interest [8]. *Concept Bottleneck Models* (CBMs) aim to define these representations based on a set of human-understandable concepts [9]. Given a task y and a set of concepts $c = \{c_j\}_{j=1}^m$, CBMs obtain a representation z and are trained to jointly optimize the two following objectives:

- (i) The task y must be perfectly solved.
- (ii) Each concept c_j must be predictable by using only a subset $z_j \in z$.

As a result of the second objective, CBMs are purported to offer: (i) increased interpretability of the representation space, and (ii) the ability to intervene a concrete concept c_j by modifying z_j and propagating these changes to the predictions.

However, a phenomenon called *information leakage* has been observed in CBMs [10, 11], whereby z encodes information that is not attributable to the defined concepts. This issue poses two major concerns: (i) it undermines the interpretability of the learned representations, and (ii) it compromises

the validity of interventions—modifying z_j may alter not only the associated concept c_j , but also other unintended factors encoded in z_j . We argue that this issue stems from a fundamental limitation in the current formulation of CBMs: the absence of an explicit Information Bottleneck (IB) [12] that actively constrains z_j to exclude information unrelated to c_j . While the second objective in CBMs encourages each z_j to retain c_j in its entirety, it does not enforce that z_j captures only information about c_j . In the worst-case scenario, z_j could encode the entire input $\mathbf{x}$ and still satisfy this objective.

We propose a solution to the aforementioned problem by introducing an IB in z_j in such a way that it does not only preserve all the information of c_j , but it also contains information only about c_j . We call these models *Minimal Concept Bottleneck Models* (MCBMs), since z_j is trained to be a *minimal sufficient* statistic of c_j [13, 14]. This contrasts with traditional CBMs, where z_j is optimized to be merely a *sufficient* statistic of c_j . In other words, while CBMs simply require that z_j preserves *all* information about c_j , MCBMs require that z_j preserves *only all* the information about c_j . We implement the IB constraint via an additional loss term derived using variational approximations. This formulation enables faithful concept interventions without violating Bayesian principles and offers greater flexibility in key choices of the model.

2 Related Work

Information Leakage in Concept Bottleneck Models This phenomenon has been observed in CBMs trained either sequentially or jointly [10, 11], and it arises when the learned representation $\mathbf{z}$ encodes information that is not present in the concept set $\mathbf{c}$. Such leakage negatively impacts both the interpretability and the intervenability of the model. As shown in [15], this issue arises when the Markovian assumption fails—i.e., when the target $\mathbf{y}$ is not fully determined by the concept set $\mathbf{c}$, or equivalently, $p(\mathbf{y}|\mathbf{c}) \neq p(\mathbf{y}|\mathbf{c}, \mathbf{x})$, which is typically the case in real-world scenarios. In [16], information leakage is tackled from the perspective of out-of-distribution generalization. Concept Embedding Models (CEMs), introduced in [17], aim to alleviate the trade-off between accuracy and interpretability commonly observed in CBMs. However, we argue that this trade-off is inherently limited by the choice of concept set, and that CEMs may, in fact, be less interpretable than standard CBMs, as they yield more entropic representations—thereby amplifying information leakage. Other CEM-based approaches [18–20] are also prone to this issue. This critique is further elaborated in [21], which introduces an information-theoretic framework along with experimental benchmarks for evaluating interpretability in the representation space. Almost contemporaneously, [22] proposes a metric for interpretability grounded in the concept of usable mutual information [23].

Information Bottleneck in Representation Learning The Information Bottleneck (IB) [12] is an information-theoretic framework that aims to balance the trade-off between preserving information about a factor variable and compressing the learned representation. A representation that retains all information about the factor is termed *sufficient*, while one that contains only information relevant to the factor—i.e., discarding all the irrelevant nuisances—is called *minimal* [24–26]. This terminology originates from the theory of minimal sufficient statistics [13, 14]. Direct computation of information-theoretic quantities is often intractable due to the complexity of the representation’s underlying distribution. To address this, variational approximations have been proposed to derive tractable evidence bounds that can be efficiently optimized [27, 28]. In this work, we leverage such approximations to optimize the IB objective with respect to the concepts.

Disentanglement A closely related topic to CBMs that has been extensively studied is *disentanglement*. Despite its popularity, there remains no universally agreed-upon definition. However, existing definitions can be broadly categorized into three groups: (i) those in [8, 29–32], which define a disentangled representation as one where changes in a specific part of the representation correspond to changes in a particular factor of variation, while remaining relatively invariant to other factors; (ii) those from works like [33, 34], which view a disentangled representation as one where modifying a single factor of variation leads to changes in only one component of the representation; and (iii) more recent perspectives [35–37], which consider a representation disentangled only if it satisfies both of the above properties. Conventional CBMs typically yield representations that align with the second definition. In contrast, MCBMs encourage representations to satisfy both criteria simultaneously by introducing an Information Bottleneck for each concept, thereby aligning with the third—and currently most widely accepted—definition of disentanglement. Our experiments demonstrate that MCBMs produce significantly more disentangled representations.

3 An Information-Theoretic Perspective on CBMs

3.1 Data Generative Process

For this scenario, we consider inputs $\mathbf{x} \in \mathcal{X}$, targets $\mathbf{y} \in \mathcal{Y}$, concepts $\mathbf{c} = \{c_j\}_{j=1}^m \in \mathcal{C}$ and nuisances $\mathbf{n} \in \mathcal{N}$, such that $p(\mathbf{x}, \mathbf{y}, \mathbf{c}, \mathbf{n}) = p(\mathbf{x}|\mathbf{c}, \mathbf{n})p(\mathbf{y}|\mathbf{x})$, i.e., the inputs $\mathbf{x}$ are described by the concepts $\mathbf{c}$ and the nuisances $\mathbf{n}$, and the targets $\mathbf{y}$ are fully described by the input $\mathbf{x}$. The only difference between $\mathbf{c}$ and $\mathbf{n}$ is that the former are observed, while the latter are not or, in other words, labels on $\mathbf{c}$ are provided to us. We assume access to a training set $\{\mathbf{x}^{(i)}, \mathbf{y}^{(i)}, \mathbf{c}^{(i)}\}_{i=1}^N$, which defines the empirical distribution $p(\mathbf{x}, \mathbf{y}, \mathbf{c}) = \sum_{i=1}^N \delta(\mathbf{x} - \mathbf{x}^{(i)}) \delta(\mathbf{y} - \mathbf{y}^{(i)}) \delta(\mathbf{c} - \mathbf{c}^{(i)})$. The graphical model corresponding to this generative process is shown in Figure 1a for the case of two concepts.

3.2 Vanilla Models

In machine learning, the most commonly studied problem is that of predicting $\mathbf{y}$ from $\mathbf{x}$, which serves as a foundation for more specialized tasks. We refer to models trained to address this problem as *Vanilla Models* (VMs). These models typically operate by first extracting an intermediate representation $\mathbf{z}$ from the input $\mathbf{x}$ via an *encoder* $p_\theta(\mathbf{z} | \mathbf{x})$. Subsequently, a prediction $\hat{\mathbf{y}} \in \mathcal{Y}$ is produced from $\mathbf{z}$ using a *task head* $q_\phi(\hat{\mathbf{y}} | \mathbf{z})$. Since $\mathbf{z}$ is intended to facilitate accurate prediction of $\mathbf{y}$, the mutual information between $\mathbf{z}$ and $\mathbf{y}$, denoted $I(\mathbf{Z}; \mathbf{Y})$, should be maximized. In Appendix B.1, we formally show that:

$$\max_{\mathbf{Z}} I(\mathbf{Z}; \mathbf{Y}) = \max_{\theta, \phi} \mathbb{E}_{p(\mathbf{x}, \mathbf{y})} [\mathbb{E}_{p_\theta(\mathbf{z}|\mathbf{x})} [\log q_\phi(\hat{\mathbf{y}}|\mathbf{z})]] \quad (1)$$

Figure 1b shows the graphical model of a Vanilla Model with a two-dimensional representation $\mathbf{z}$. Black edges represent the encoder $p_\theta(\mathbf{z} | \mathbf{x})$, while green edges indicate the task head $q_\phi(\hat{\mathbf{y}} | \mathbf{z})$.

The *encoder* is typically chosen to be deterministic, i.e., $p_\theta(\mathbf{z} | \mathbf{x}) = \delta(\mathbf{z} - f_\theta(\mathbf{x}))$. However, for tractability reasons (see Section 4.2), we adopt a stochastic formulation where $p_\theta(\mathbf{z} | \mathbf{x}) = \mathcal{N}(\mathbf{z}; f_\theta(\mathbf{x}), \sigma_x^2 \mathbf{I})$, as summarized in Table 1. The choice of *task head* $q_\phi(\hat{\mathbf{y}} | \mathbf{z})$ depends on the structure of the output space $\mathcal{Y}$, also detailed in Table 1. The objective in Equation (1) corresponds to minimizing the cross-entropy loss when $\mathbf{y}$ is binary or multiclass, and to minimizing the mean squared error between $\mathbf{y}$ and $g_\phi^y(\mathbf{z})$ when $\mathbf{y}$ is continuous.

3.3 Concept Bottleneck Models

Vanilla Models generally lack interpretability with respect to the known concepts $\mathbf{c}$, as the encoder f_θ is often opaque and difficult to analyze. Moreover, these models tend to entangle the concepts in such a way that it becomes intractable to determine how individual concepts influence specific components of the latent representation $\mathbf{z}$, and consequently the predictions $\hat{\mathbf{y}}$. *Concept Bottleneck Models* (CBMs) have been introduced to address this limitation. In a CBM, each concept c_j is predicted from a dedicated latent representation $\mathbf{z}_j$ via a *concept head* $q(\hat{c}_j | \mathbf{z}_j)$. Consequently, $\mathbf{z}_j$ must encode all information about c_j —that is, $\mathbf{z}_j$ must be a *sufficient* representation for c_j . This requirement can be formalized as maximizing the mutual information $I(\mathbf{Z}_j; \mathbf{C}_j)$, for which the following identity—proved in Appendix B.2—is employed:

$$\max_{\mathbf{Z}_j} I(\mathbf{Z}_j; \mathbf{C}_j) = \max_{\theta, \phi} \mathbb{E}_{p(\mathbf{x}, \mathbf{c}_j)} [\mathbb{E}_{p_\theta(\mathbf{z}_j|\mathbf{x})} [\log q_\phi(\hat{c}_j|\mathbf{z}_j)]] \quad (2)$$

As illustrated in Figure 1c, CBMs extend Vanilla Models by incorporating a concept head $q(\hat{c}_j | \mathbf{z}_j)$, depicted with blue arrows. These models jointly optimize the objectives in Equations (1) and (2).

As detailed in Table 1, the form of the concept head $q_\phi(\hat{c}_j | \mathbf{z}_j)$ depends on the nature of the concept space $\mathcal{C}$. The objective in Equation (2) corresponds to minimizing the cross-entropy loss when c_j is binary or multiclass, and the mean squared error between c_j and $g_\phi^c(\mathbf{z}_j)$ when c_j is continuous.

Table 1: Distributions considered in this work. f_θ is typically modeled by a large neural network while g_ϕ^y , g_ϕ^c and g_ϕ^z by simpler neural networks. We consider $\mathbf{z}$ to be always continuous.

	$p_\theta(\mathbf{z} \mathbf{x})$	$q_\phi(\hat{\mathbf{y}} \mathbf{z})$	$q_\phi(\hat{c}_j \mathbf{z}_j)$	$q_\phi(\hat{z}_j \mathbf{c}_j)$
Binary	-	Bernoulli $\left(g_\phi^y(\mathbf{z})\right)$	Bernoulli $\left(g_\phi^c(\mathbf{z}_j)\right)$	-
Multiclass	-	Categorical $\left(g_\phi^y(\mathbf{z})\right)$	Categorical $\left(g_\phi^c(\mathbf{z}_j)\right)$	-
Continuous	$\mathcal{N}(f_\theta(\mathbf{x}), \sigma_x^2 \mathbf{I})$	$\mathcal{N}(g_\phi^y(\mathbf{z}), \sigma_y^2 \mathbf{I})$	$\mathcal{N}(g_\phi^c(\mathbf{z}_j), \sigma_c^2 \mathbf{I})$	$\mathcal{N}(g_\phi^z(\mathbf{c}_j), \sigma_z^2 \mathbf{I})$

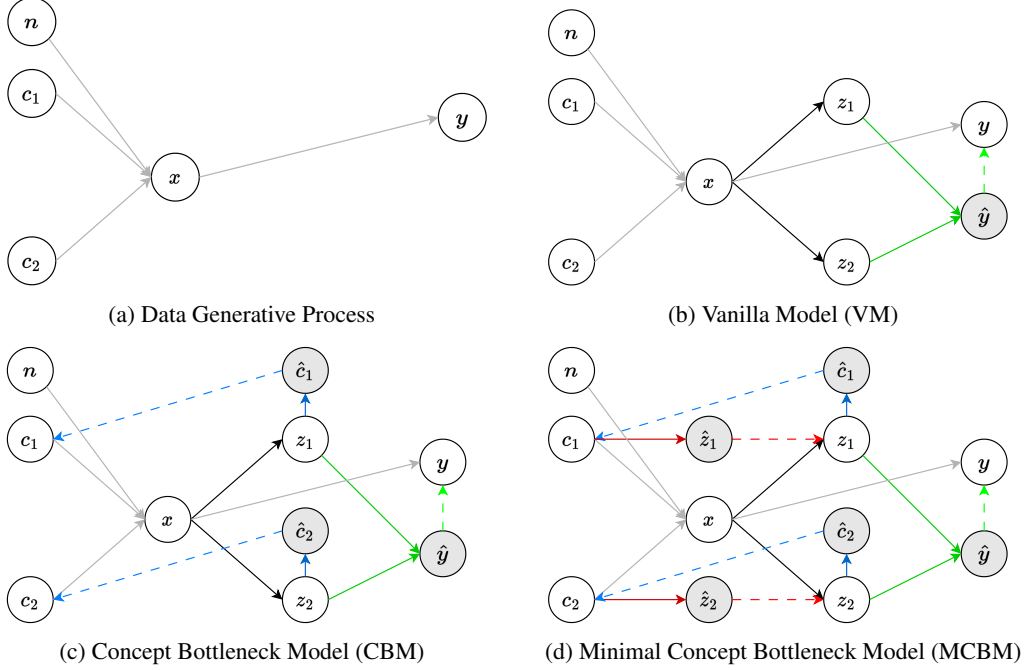


Figure 1: Graphical models of the different systems described. We consider two concepts and two-dimensional representations. We provide in Appendix A the analogous figure for m concepts and m -dimensional representations. Inputs $\mathbf{x}$ are defined by some concepts $\{c_j\}_{j=1}^m$ and nuisances $\mathbf{n}$; and targets $\mathbf{y}$ are defined by $\mathbf{x}$ (gray arrows). Vanilla models obtain the representations $\{z_j\}_{j=1}^m$ from $\mathbf{x}$ through the *encoder* $p_\theta(z|\mathbf{x})$ (black arrows) and solve the task $\hat{\mathbf{y}}$ sequentially through the *task head* $q_\phi(\mathbf{y}|\mathbf{x})$ (green arrows). Concept Bottleneck Models try to make a prediction $\hat{c}_j$ of each concept c_j from one representation z_j through the *concept head* $q(\hat{c}_j|z_j)$ (blue arrows). Minimal CBMs (ours) try to make a prediction $\hat{z}_j$ of each representation z_j from one concept c_j through the *representation head* $q(z_j|c_j)$ (red arrows).

4 On the Lack of a Bottleneck in CBMs

As explained in Section 1, one of the main advantages commonly attributed to CBMs over vanilla models is their ability to perform concept-level interventions and propagate the effects to the model’s predictions. For instance, consider a scenario in which we aim to estimate $p(\hat{\mathbf{y}} | c_j = \alpha, \mathbf{x})$ —that is, the model’s predicted output $\hat{\mathbf{y}}$ had the concept c_j been set to the value α . In CBMs, this intervention is typically performed through the latent representation z_j , according to:

$$p(\hat{\mathbf{y}} | c_j = \alpha, \mathbf{x}) = \iint p(\hat{\mathbf{y}} | z_j, \mathbf{z}_{\setminus j}) p(z_j | c_j = \alpha) p(\mathbf{z}_{\setminus j} | \mathbf{x}) dz_j d\mathbf{z}_{\setminus j} \quad (3)$$

However, in the original formulation of CBMs (described in Section 3.3), the conditional distribution $p(z_j | c_j)$ is not defined—there is no directed path from c_j to z_j in Figure 1c. Intuitively, because z_j may encode information about $\mathbf{x}$ beyond c_j , it cannot be fully determined by c_j alone. This leads to a key question: *how can interventions be performed in CBMs if $p(z_j | c_j)$ is unknown?* In what follows, we address this question and argue that current practices often violate basic Bayesian principles, leading to inconsistencies and limiting the architectural flexibility of the model. To resolve this, we introduce *Minimal CBMs* (MCBMs), a theoretically grounded framework that addresses these issues and enables principled interventions.

4.1 Intervention Mechanisms in Concept Bottleneck Models

As previously discussed, the conditional distribution $p(z_j | c_j)$ is not explicitly defined in standard CBMs. To enable a mechanism for performing interventions, two constraints are typically imposed:

- (i) Concepts c_j , for $j = 1, 2, \dots, m$, are assumed to be binary. If a concept is originally multiclass with k categories, it is converted into k binary concepts (One-vs-Rest [38]).
- (ii) The *concept head* is defined as $q_\phi(c_j | z_j) = \sigma(z_j)$, where σ denotes the sigmoid function.

Given that σ is an invertible function, this setup permits defining $p(z_j | c_j) = \sigma^{-1}(c_j)$. However, since $\sigma^{-1}(1) = -\sigma^{-1}(0) = \infty$, this inversion becomes ill-defined at the binary extremes. To address this, in practice, $p(z_j | c_j = 0)$ and $p(z_j | c_j = 1)$ are approximated as the 5th and 95th percentiles of the empirical distribution of z_j , respectively [9]. Nevertheless, this practice introduces two issues:

1. One-vs-Rest strategies exhibit the following limitations: (i) individual binary classifiers tend to be biased toward the negative class, (ii) multiple classifiers may simultaneously assign a "positive" label to the same input, and (iii) the predicted probabilities across classifiers are typically uncalibrated [39]. These limitations are further illustrated in Section 5.3.
2. The intervention procedure based on the sigmoid inverse does not adhere to Bayes' rule, i.e., $p(z_j | c_j) = \frac{q_\phi(c_j|z_j)p(z_j)}{p(c_j)}$. While $p(c_j)$ can be approximated using its empirical distribution, the marginal $p(z_j)$ remains unknown and is challenging to estimate.

4.2 Minimal Concept Bottleneck Models

As previously discussed, CBMs lack an explicit mechanism for enforcing a bottleneck, which often undermines their primary advantages. To address this limitation, we introduce *Minimal Concept Bottleneck Models* (MCBMs), which explicitly impose an *Information Bottleneck*. This ensures that each representation z_j not only retains all information about the associated concept c_j , but also contains information *only* about c_j . In other words, z_j becomes a *minimal sufficient* representation of c_j . To achieve this, we introduce a *representation head* $q_\phi(\hat{z}_j | c_j)$ that predicts z_j from c_j . This encourages z_j to discard information unrelated to c_j , as doing so improves the predictive accuracy of $q_\phi(\hat{z}_j | c_j)$. Formally, this objective corresponds to minimizing the conditional mutual information $I(Z_j; X | C_j)$. We note that if z_j is a representation of $\mathbf{x}$, then the following propositions are equivalent: (i) $I(Z_j; X | C_j) = 0$, (ii) the Markov Chain $X \leftrightarrow C_j \leftrightarrow Z_j$ is satisfied and (iii) $p(z_j | c_j) = p(z_j | \mathbf{x})$. To minimize $I(Z_j; X | C_j)$, we leverage the following identity, which is proven in Appendix B.3:

$$\min_{Z_j} I(Z_j; X | C_j) = \min_{\theta, \phi} \mathbb{E}_{p(\mathbf{x}, c_j)} [D_{KL}(p_\theta(z_j | \mathbf{x}) || q_\phi(\hat{z}_j | c_j))] \quad (4)$$

Figure 1d illustrates how MCBMs extend CBMs by introducing the representation head $q(\hat{z}_j | c_j)$, depicted with red arrows. These models are trained to jointly optimize the objectives given in Equations (1), (2), and (4). Although the KL Divergence in Equation (4) does not admit a closed-form solution in general, we show in Appendix B.4 that, under the distributional assumptions listed in Table 1, it reduces to the mean squared error between $f_\theta(\mathbf{x})$ and $g_\phi^z(c_j)$.

4.3 Practical Considerations for Optimizing MCBMs

To optimize MCBMs, we combine the three previously introduced objectives, incorporating two key considerations. First, we approximate the expectations over $p(\mathbf{x}, \mathbf{y})$ and $p(\mathbf{x}, c_j)$ using the empirical data distribution described in Section 3.1, replacing integrals with summations over the dataset. Second, to enable gradient-based optimization through the stochastic encoder, we apply the reparameterization trick [40], allowing for low-variance gradient estimates during back-propagation. $\mathbb{E}_{p_\theta(\mathbf{z}|\mathbf{x})} [\log q_\phi(\mathbf{y}|\mathbf{z})] \approx \sum_i \log q_\phi(\mathbf{y} | f'_\theta(\mathbf{x}, \epsilon^{(i)}))$ and $E_{p_\theta(z_j|\mathbf{x})} [\log q_\phi(\hat{c}_j|z_j)] \approx \sum_i \log q_\phi(\hat{c}_j | f'_{\theta,j}(\mathbf{x}, \epsilon^{(i)}))$, where $f'_\theta(\mathbf{x}, \epsilon) = f_\theta(\mathbf{x}) + \sigma_x^2 I \epsilon$ (due to the choice of $p_\theta(\mathbf{z}|\mathbf{x})$ in Table 1), $\epsilon \sim \mathcal{N}(0, I)$ and $f'_{\theta,j}(\mathbf{x}, \epsilon)$ corresponds to the element j of $f'_\theta(\mathbf{x}, \epsilon)$. Combining the considerations above yields the final objective for training MCBMs, as shown in Equation (5), where β and γ are hyperparameters. The first term corresponds to the objective used in Vanilla Models, the second term is introduced in CBMs, and the third is specific to MCBMs. Detailed training algorithms for the various cases listed in Table 1 are provided in Appendix C.

$$\begin{aligned} \max_{\theta, \phi} \sum_{k=1}^N \sum_i \log q_\phi(\hat{\mathbf{y}} | f'_\theta(\mathbf{x}^{(k)}, \epsilon^{(i)})) &+ \beta \sum_{j=1}^n \log q_\phi(\hat{c}_j | f'_{\theta,j}(\mathbf{x}^{(k)}, \epsilon^{(i)})) \\ &- \gamma \sum_{j=1}^n D_{KL}(p_\theta(z_j | \mathbf{x}^{(k)}) || q_\phi(\hat{z}_j | c_j^{(k)})) \end{aligned} \quad (5)$$

4.4 Intervention Mechanisms in Minimal Concept Bottleneck Models

As discussed at the beginning of this section, intervention procedures require access to $p(z_j | c_j)$. As outlined in Section 4.1, this dependence anchors the intervention process to specific architectural design choices in CBMs. Moreover, such procedures violate Bayesian principles, as the prior distribution $p(z_j)$ is typically unknown. In MCBMs, since z_j is explicitly designed to contain information only about c_j , modifying z_j corresponds to intervening solely on c_j . This is reflected in the graphical model illustrated in Figure 1d, where MCBMs introduce a directed path from c_j to z_j through the intermediate variable $\hat{z}_j$. This structure permits the computation of:

$$p(z_j | c_j) = \int p(z_j | \hat{z}_j) q_\phi(\hat{z}_j | c_j) d\hat{z}_j \quad (6)$$

which simplifies to $p(z_j | c_j) = q_\phi(z_j | c_j)$ when $p(z_j | \hat{z}_j) = \delta(z_j - \hat{z}_j)$, which holds when the objective in Equation (4) is optimized to convergence—or, equivalently, when z_j contains information exclusively about c_j . As shown in Table 1, we define the encoder distribution as $q_\phi(z_j | c_j) = \mathcal{N}(g_\phi^z(c_j), \sigma_z^2 I)$. The mean function $g_\phi^z(c_j)$ is chosen according to the following rules:

- (i) For binary concepts, $g_\phi^z(c_j) = \lambda$ if $c_j = 1$, and $g_\phi^z(c_j) = -\lambda$ otherwise.
- (ii) For categorical concepts, $g_\phi^z(c_j) = \lambda \cdot \text{one_hot}(c_j)$. This mirrors the *Prototypical Learning* [41] approach where class-dependent prototypes $\{g_\phi^z(c_j)\}_{j=1}^m$ are fixed, as adopted in [42].
- (iii) For continuous concepts, $g_\phi^z(c_j) = \lambda \cdot c_j$.

Here, λ is a scaling constant that controls the norm of the latent representation and is fixed to $\lambda = 3$ across all experiments. Regarding the variance term, we set: (i) $\sigma_x = 0$ in the case of CBMs to obtain a deterministic encoder in line with their original formulation, and (ii) $\sigma_x = \sigma_z = 1$ for MCBMs.

5 Experiments

In this section, we present a series of experiments designed to empirically demonstrate that CBMs fail to impose an effective bottleneck, even in simple settings. We also examine the consequences of this limitation. In contrast, we show that MCBMs successfully enforce a bottleneck, thereby mitigating the issues that arise in its absence. Additional implementation details, including encoder architectures and training hyperparameters for each experiment, are provided in Appendix D.

5.1 Do CBMs and MCBMs Leak Information?

In some works, it is assumed that the task y is fully defined by the concepts c . However, it is unrealistic to have a finite set of concepts that we understand as humans and that totally describe an arbitrarily complex task. In general, some of the nuisances n that describe the input x will also influence y . Specifically, we decompose the nuisances as $n = \{n_y, n_{\bar{y}}\}$, where n_y captures nuisances that, together with the concepts, describe y , while $n_{\bar{y}}$ comprises those that are independent of y (see Figure 2a). As shown through this text, CBMs have no incentive to remove information that is not related to the concepts. Additionally, since the objective of models in general is to solve y , they have incentives to preserve not only c but also n_y in their representations. Although this may lead to improved performance in solving the task, it comes at the expense of interpretability and the ability to intervene in the system. For example if $z_j \in z$ were intended to represent the concept $c_j \in c$, then one could believe that an intervention is made solely on c_j by modifying z_j . However z_j could also contain information on n_y (or even $n_{\bar{y}}$, as we see later), so we would be intervening also the nuisances by modifying z_j , thereby invalidating any causal conclusion drawn. By introducing the Information Bottleneck in equation (4), we force the model to remove the nuisances from z_j , thus restoring the validity of any causal analysis about c_j . We note that this will necessarily negatively impact the performance in solving the task y by the nature of the problem. If c is incomplete and information from n is necessary to solve y (i.e., $n_{\bar{y}} \neq \emptyset$), the performance will decrease if z does not contain information about n . We analyze next if n_y and $n_{\bar{y}}$ are present in the representation for different versions of CBMs and datasets.

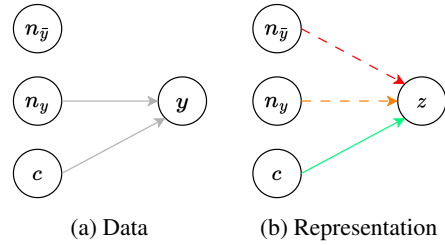
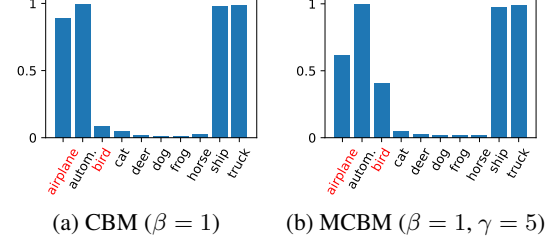
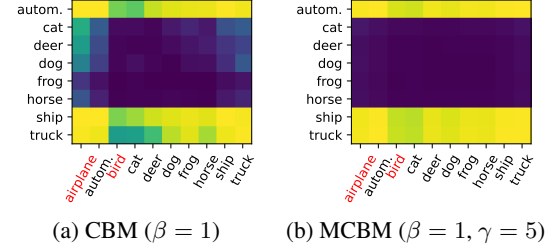


Figure 2: Some nuisances $n_y \in n$ affect the task y while others $n_{\bar{y}} \in n$ do not. None of them should affect the representation z since it must be fully described by the concepts c .

Table 2: Average value of URR for task-related nuisances $\mathbf{n}_y$.

	CIFAR-10	MPI3D	Shapes3D	CelebA	CUB
Vanilla	33.5 ± 4.3	35.0 ± 1.9	45.5 ± 6.4	10.7 ± 3.5	3.8 ± 1.0
CBM	34.7 ± 0.2	28.1 ± 0.5	8.1 ± 2.9	11.5 ± 3.5	3.8 ± 0.8
CEM	35.1 ± 0.1	43.2 ± 5.2	15.8 ± 3.9	16.8 ± 3.3	3.9 ± 1.1
MCBM (low γ)	24.8 ± 0.6	10.7 ± 0.1	2.5 ± 0.1	4.3 ± 0.0	3.4 ± 0.9
MCBM (medium γ)	19.1 ± 1.0	6.7 ± 0.2	0.2 ± 0.3	7.9 ± 2.9	2.8 ± 0.8
MCBM (high γ)	11.0 ± 0.5	0.0 ± 0.0	0.0 ± 0.0	8.0 ± 2.4	2.4 ± 1.0

Task-related information leakage We analyze whether $\mathbf{n}_y$ is encoded in $\mathbf{z}$ —that is, whether the dependency illustrated by the orange arrow in Figure 2b is present—and examine its consequences. To illustrate this, we design a simple experiment based on the CIFAR-10 dataset [43]. Since all classes in CIFAR-10 correspond to either *animals* or *vehicles*, we define the target $\mathbf{y}$ as the binary task of distinguishing between these two categories. The concepts $\mathbf{c}$ are defined using all classes except *airplane* and *bird*. Under a proper bottleneck, the model should be unable to accurately predict $\mathbf{y}$ for these excluded classes, as their information is not captured by any concept. Figure 3 shows the mean predictions $\hat{y}$ for each class, comparing a CBM and an MCBM. The CBM confidently predicts $\mathbf{y}$ even for the excluded classes, indicating a substantial degree of information leakage. In contrast, the MCBM significantly mitigates this issue, displaying low confidence for excluded classes while maintaining high confidence for those included in the concept set. To assess the impact on interventions, Figure 4 shows model predictions after simulating an intervention corresponding to each concept. In the CBM, predictions following interventions exhibit reduced confidence for certain classes—an effect attributed to residual nuisance information in the representation. This effect is notably reduced in the MCBM.

Figure 3: Average prediction $\hat{y}$ in animal (0) vs. vehicle (1) classification for the different classes when excluding *airplane* and *bird* from the concept set.Figure 4: Average prediction $\hat{y}$ for *animal* (purple) vs. *vehicle* (yellow) when intervening on $\mathbf{z}$ to match the x-axis class instead of the true class (y-axis).

To systematically analyze this phenomenon, we consider several datasets and define distinct task–concept configurations for each. Specifically: (i) In CIFAR-10 [43], we adopt the described scenario: $\mathbf{y}$ corresponds to a binary classification between *animals* and *vehicles*, $\mathbf{n}_y$ includes *airplane* and *bird*, and $\mathbf{c}$ represents the remaining classes. (ii) In MPI3D [44], $\mathbf{y}$ corresponds to the *object shape*, $\mathbf{n}_y$ to the *horizontal axis*, $\mathbf{n}_{\bar{y}}$ to the *vertical axis*, and $\mathbf{c}$ to the remaining generative factors. (iii) In Shapes3D [30], $\mathbf{y}$ is the *shape*, $\mathbf{n}_y$ includes the *floor color* and *wall color*, $\mathbf{n}_{\bar{y}}$ is the *orientation*, and $\mathbf{c}$ includes the remaining factors. (iv) In CelebA [45], $\mathbf{y}$ is *gender*, $\mathbf{c}$ consists of eight randomly selected attributes, and $\mathbf{n}_y$ includes the remaining factors, all of which are correlated with $\mathbf{y}$. (v) In CUB [46], $\mathbf{y}$ corresponds to the *bird species*, $\mathbf{c}$ includes the concepts from seven randomly selected attribute groups, and $\mathbf{n}_y$ comprises the attributes from the remaining 20 groups. Further details for each setup are provided in Appendix D. We recall that, while our MCBMs natively support multiclass concepts, the other models evaluated in this work are limited to binary concepts. Therefore, to ensure fair comparisons, factors with m classes are represented as m separate binary concepts.

To assess the presence of a nuisance factor $n_j \in \mathbf{n}_y$ in $\mathbf{z}$, we aim to estimate $I(N_j; \mathbf{Z} | \mathbf{C})$ —that is, the amount of additional information that $\mathbf{z}$ provides about n_j beyond what is already captured by $\mathbf{c}$. Since this quantity is intractable in practice, we approximate it as $\hat{I}(N_j; \mathbf{Z} | \mathbf{C}) = \hat{H}(N_j | \mathbf{C}) - \hat{H}(N_j | \mathbf{C}, \mathbf{Z})$, where $\hat{H}(N_j | \mathbf{C}) = -\sum_{k=1}^N \log h_{\psi}^c(\mathbf{c}^{(k)})$ and $\hat{H}(N_j | \mathbf{C}, \mathbf{Z}) = -\sum_{k=1}^N \log h_{\psi}^{cz}(\mathbf{c}^{(k)}, \mathbf{z}^{(k)})$. Here, h_{ψ}^c and h_{ψ}^{cz} are MLP classifiers trained to predict n_j from $\mathbf{c}$

and (c, z) , respectively. In Table 2, we report the average value of $\frac{\hat{I}(N_j; Z|C)}{H(N_j)}$ across all $n_j \in \mathbf{n}_y$, which we refer to as the *Uncertainty Reduction Ratio* (URR). From these results, we draw the following conclusions: (i) regular CBMs do not consistently remove more nuisance information than VMs—in fact, contrary to common belief, CBMs can sometimes impose a weaker bottleneck than VMs; (ii) CEMs representations are those with more nuisances, i.e., they always introduce a weaker bottleneck even than VMs; and (iii) MCBMs are the models removing more nuisances from the representation, especially when γ increases.

Task-unrelated information leakage On the one hand, neural networks typically discard information from the input, as their layers are generally non-invertible [47, 48]. On the other hand, there is no incentive to retain $\mathbf{n}_{\bar{y}}$ in the representation z , since it is irrelevant for predicting y . From this, one might expect that z should not contain information about $\mathbf{n}_{\bar{y}}$. However, prior work has shown that neural representations often retain information that is not directly relevant to the target task [24, 49]. To investigate this phenomenon, we analyze if $\mathbf{n}_{\bar{y}}$ is present in z for MPI3D and Shapes3D—the only settings in our experimental setup where $\mathbf{n}_{\bar{y}}$ is non-empty. We report the URR values for the nuisance variables in $\mathbf{n}_{\bar{y}}$ in Table 3, and observe the following: (i) in VMs, CBMs, and CEMs, nuisance variables remain present in the representation, even when they are irrelevant to the task y ; and (ii) MCBMs consistently eliminate task-irrelevant nuisances across all values of γ , which is expected: since there is no incentive to preserve $\mathbf{n}_{\bar{y}}$, and MCBMs explicitly encourage its removal, such information tends to be discarded.

Table 3: Average URR for task-unrelated nuisances.

	MPI3D	Shapes3D
Vanilla	11.3 ± 0.1	42.7 ± 9.1
CBM	7.4 ± 1.9	20.6 ± 3.3
CEM	15.5 ± 4.2	40.9 ± 1.8
MCBM (low γ)	0.0 ± 0.0	0.0 ± 0.0
MCBM (medium γ)	0.0 ± 0.0	0.0 ± 0.0
MCBM (high γ)	0.0 ± 0.0	0.0 ± 0.0

Are concepts removed to a greater extent in MCBMs?

One might suspect that removing nuisance information from the representation could inadvertently eliminate information about the concepts as well. To investigate this, we report concept prediction accuracy for CelebA and CUB in Table 4 (similar trends were observed on the remaining datasets). We observe no significant overall differences across models: while MCBMs show slightly lower concept accuracy on CelebA, they outperform the baselines on CUB for lower γ . A regular trend, however, is that increasing the hyperparameter γ in MCBMs leads to a gradual decrease in concept accuracy, which is consistent with our expectations—stronger regularization may inadvertently suppress features that correlate with the concepts. These results suggest that MCBMs are effective at removing nuisance information while largely preserving the concept-relevant content in the representation.

Table 4: Average concepts accuracy

	CelebA	CUB
CBM	92.2 ± 0.7	96.5 ± 0.1
CEM	92.1 ± 0.8	96.5 ± 0.1
MCBM (low γ)	92.0 ± 0.8	96.7 ± 0.1
MCBM (medium γ)	92.1 ± 1.0	96.4 ± 0.1
MCBM (high γ)	91.8 ± 1.1	95.8 ± 0.4

5.2 Do MCBMs Yield More Interpretable Representations?

One of the key properties often attributed to CBMs is that their internal representations align with human-interpretable concepts. While this generally holds, we show that MCBMs yield representations that are even more interpretable. To support this claim, we employ two metrics: (i) *Centered Kernel Alignment* (CKA) [50–52], computed between the learned representations and the concept labels (encoded as one-hot vectors), which quantifies the similarity between representation spaces using kernel methods; and (ii) *Disentanglement* [36], which assesses whether each dimension of the representation z_j captures at most one concept c_j . We argue that these metrics offer meaningful evidence of interpretability because (i) alignment between the representations and the concepts indicates whether the information is organized in a concept-aware

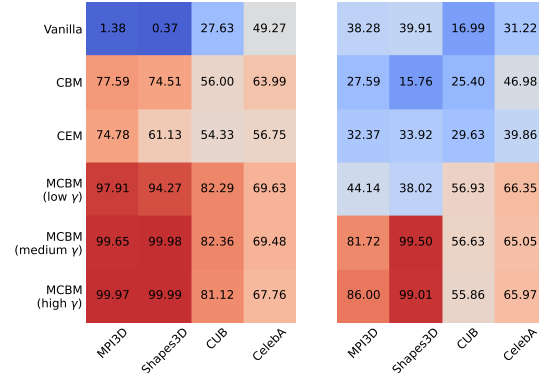


Figure 5: CKA (left) and Disentanglement (right).

fashion; and (ii) disentanglement reflects the extent to which individual concepts are independently encoded in the learned representations. In Figure 5, we observe that: (i) although the representations learned by CBMs and CEMs exhibit substantially higher alignment with the concepts than VMs, their disentanglement levels are not reliably superior; and (ii) MCBMs consistently achieve improvements in both alignment and disentanglement. Thus, we conclude that removing nuisances from the representations leads to more interpretable representations.

5.3 What Are the Pitfalls of Binarizing Multiclass Concepts?

As discussed in Section 4.1, CBMs require binarizing multiclass concepts to support interventions—i.e., they adopt a One-vs-Rest classification scheme. In contrast, as discussed in Section 4.4, MCBMs provide a natural and principled framework for modeling and intervening on multiclass concepts directly. Binarizing multiclass variables can introduce several issues, particularly in the presence of class imbalance, which is common in real-world applications. To illustrate this limitation, we design an experiment using a four-class spiral dataset with imbalanced class distributions. We train two CBMs for comparison: (i) one that binarizes concepts using a One-vs-Rest approach, and (ii) one that models concepts directly as multiclass variables.

First, as shown in Figure 6, the One-vs-Rest model tends to favor the most frequent class more often than the Multiclass model, particularly when the true class has low prevalence and is similar (i.e., spatially close in the spiral setting) to the dominant class. Second, Figure 7 illustrates that, in the Multiclass model, the predicted class likelihood increases gradually from regions far from the decision boundaries toward the boundaries, resulting in a well-calibrated probability distribution. Such patterns are absent in the One-vs-Rest model. Finally, in Figure 8, we observe that the second-highest predicted likelihood in the One-vs-Rest model is close to one in certain regions—indicating that multiple classes are simultaneously assigned high confidence. In contrast, the Multiclass model assigns low second-highest likelihoods in regions far from the boundaries, with values increasing only near the decision boundaries.

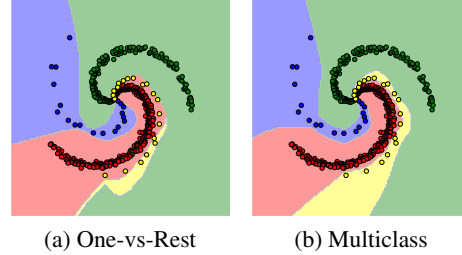


Figure 6: Class Boundaries

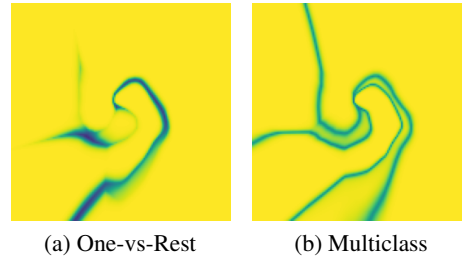


Figure 7: Highest likelihood

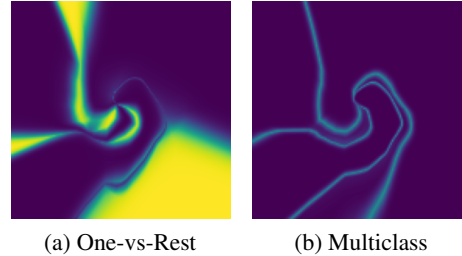


Figure 8: Second highest likelihood

6 Conclusions

In this paper, we argue that—contrary to common belief—Concept Bottleneck Models (CBMs) do not enforce a true bottleneck: while representations are encouraged to retain concept-related information, they are not constrained to discard nuisance information. This limitation results in a lack of theoretical guarantees for intervention procedures and reduced interpretability in the representation space. To address this, we propose Minimal Concept Bottleneck Models (MCBMs), which introduce an Information Bottleneck in the representation space via an additional loss term derived through variational approximations. Beyond establishing a proper bottleneck, our formulation ensures that interventions do not violate Bayesian principles and introduces greater design flexibility compared to standard CBMs. Through empirical analysis, we show that CBMs—and their existing variants—fail to effectively remove nuisance information from the representation, even when such information is irrelevant to the target task. In contrast, MCBMs successfully eliminate nuisances while preserving concept-relevant information, resulting in more interpretable representations and principled intervention mechanisms. Finally, we highlight a common limitation of CBMs: the need to binarize multiclass concepts due to architectural constraints. MCBMs natively support multiclass concepts, thereby overcoming the drawbacks introduced by this binarization process.

Acknowledgments and Disclosure of Funding

This work has received funding from the European Union’s Horizon 2020 research and innovation programme under the Marie Skłodowska-Curie grant agreement No 101007666, MCIN/AEI/10.13039/501100011033 under Grant PID2021-126061OB-C44, and the Government of Aragón (Grant Group T36 23R).

References

- [1] Muhammad Aurangzeb Ahmad, Carly Eckert, and Ankur Teredesai. Interpretable machine learning in healthcare. In *Proceedings of the 2018 ACM international conference on bioinformatics, computational biology, and health informatics*, pages 559–560, 2018.
- [2] Yao Xie, Melody Chen, David Kao, Ge Gao, and Xiang’Anthony’ Chen. Chexplain: enabling physicians to explore and understand data-driven, ai-enabled medical imaging analysis. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems*, pages 1–13, 2020.
- [3] Di Jin, Elena Sergeeva, Wei-Hung Weng, Geeticka Chauhan, and Peter Szolovits. Explainable deep learning in healthcare: A methodological survey from an attribution view. *WIREs Mechanisms of Disease*, 14(3):e1548, 2022.
- [4] Damiano Brigo, Xiaoshan Huang, Andrea Pallavicini, and Hartz Saez de Ocariz Borde. Interpretability in deep learning for finance: a case study for the heston model. *arXiv preprint arXiv:2104.09476*, 2021.
- [5] Shun Liu, Kexin Wu, Chufeng Jiang, Bin Huang, and Danqing Ma. Financial time-series forecasting: Towards synergizing performance and interpretability within a hybrid machine learning approach. *arXiv preprint arXiv:2401.00534*, 2023.
- [6] Jinkyu Kim and John Canny. Interpretable learning for self-driving cars by visualizing causal attention. In *Proceedings of the IEEE international conference on computer vision*, pages 2942–2950, 2017.
- [7] Zhenhua Xu, Yujia Zhang, Enze Xie, Zhen Zhao, Yong Guo, Kwan-Yee K Wong, Zhenguo Li, and Hengshuang Zhao. Drivegpt4: Interpretable end-to-end autonomous driving via large language model. *IEEE Robotics and Automation Letters*, 2024.
- [8] Yoshua Bengio, Aaron Courville, and Pascal Vincent. Representation learning: A review and new perspectives. *IEEE transactions on pattern analysis and machine intelligence*, 35(8):1798–1828, 2013.
- [9] Pang Wei Koh, Thao Nguyen, Yew Siang Tang, Stephen Mussmann, Emma Pierson, Been Kim, and Percy Liang. Concept bottleneck models. In *International conference on machine learning*, pages 5338–5348. PMLR, 2020.
- [10] Andrei Margeloiu, Matthew Ashman, Umang Bhatt, Yanzhi Chen, Mateja Jamnik, and Adrian Weller. Do concept bottleneck models learn as intended? *arXiv preprint arXiv:2105.04289*, 2021.
- [11] Anita Mahinpei, Justin Clark, Isaac Lage, Finale Doshi-Velez, and Weiwei Pan. Promises and pitfalls of black-box concept learning models. *arXiv preprint arXiv:2106.13314*, 2021.
- [12] Naftali Tishby, Fernando C Pereira, and William Bialek. The information bottleneck method. *arXiv preprint physics/0004057*, 2000.
- [13] Ronald A Fisher. On the mathematical foundations of theoretical statistics. *Philosophical transactions of the Royal Society of London. Series A, containing papers of a mathematical or physical character*, 222 (594-604):309–368, 1922.
- [14] Ronald A Fisher. The logic of inductive inference. *Journal of the royal statistical society*, 98(1):39–82, 1935.
- [15] Marton Havasi, Sonali Parbhoo, and Finale Doshi-Velez. Addressing leakage in concept bottleneck models. *Advances in Neural Information Processing Systems*, 35:23386–23397, 2022.
- [16] Emanuele Marconato, Andrea Passerini, and Stefano Teso. Glancenets: Interpretable, leak-proof concept-based models. *Advances in Neural Information Processing Systems*, 35:21212–21227, 2022.
- [17] Mateo Espinosa Zarlenga, Pietro Barbiero, Gabriele Ciravegna, Giuseppe Marra, Francesco Giannini, Michelangelo Diligenti, Zohreh Shams, Frederic Precioso, Stefano Melacci, Adrian Weller, et al. Concept embedding models: Beyond the accuracy-explainability trade-off. *Advances in Neural Information Processing Systems*, 35:21400–21413, 2022.

- [18] Mateo Espinosa Zarlenga, Katie Collins, Krishnamurthy Dvijotham, Adrian Weller, Zohreh Shams, and Mateja Jamnik. Learning to receive help: Intervention-aware concept embedding models. *Advances in Neural Information Processing Systems*, 36:37849–37875, 2023.
- [19] Lijie Hu, Tianhao Huang, Huanyi Xie, Xilin Gong, Chenyang Ren, Zhengyu Hu, Lu Yu, Ping Ma, and Di Wang. Semi-supervised concept bottleneck models. *arXiv preprint arXiv:2406.18992*, 2024.
- [20] Divyansh Srivastava, Ge Yan, and Lily Weng. Vlg-cbm: Training concept bottleneck models with vision-language guidance. *Advances in Neural Information Processing Systems*, 37:79057–79094, 2024.
- [21] Enrico Parisini, Tapabrata Chakraborti, Chris Harbron, Ben D MacArthur, and Christopher RS Banerji. Leakage and interpretability in concept-based models. *arXiv preprint arXiv:2504.14094*, 2025.
- [22] Mikael Makonnen, Moritz Vandenheide, Sonia Laguna, and Julia E Vogt. Measuring leakage in concept-based methods: An information theoretic approach. *arXiv preprint arXiv:2504.09459*, 2025.
- [23] Yilun Xu, Shengjia Zhao, Jiaming Song, Russell Stewart, and Stefano Ermon. A theory of usable information under computational constraints. *arXiv preprint arXiv:2002.10689*, 2020.
- [24] Alessandro Achille and Stefano Soatto. Emergence of invariance and disentanglement in deep representations. *Journal of Machine Learning Research*, 19(50):1–34, 2018.
- [25] Alessandro Achille and Stefano Soatto. Information dropout: Learning optimal representations through noisy computation. *IEEE transactions on pattern analysis and machine intelligence*, 40(12):2897–2905, 2018.
- [26] Ravid Shwartz Ziv and Yann LeCun. To compress or not to compress—self-supervised learning and information theory: A review. *Entropy*, 26(3):252, 2024.
- [27] Alexander A Alemi, Ian Fischer, Joshua V Dillon, and Kevin Murphy. Deep variational information bottleneck. *arXiv preprint arXiv:1612.00410*, 2016.
- [28] Ian Fischer. The conditional entropy bottleneck. *Entropy*, 22(9):999, 2020.
- [29] Irina Higgins, Loic Matthey, Arka Pal, Christopher P Burgess, Xavier Glorot, Matthew M Botvinick, Shakir Mohamed, and Alexander Lerchner. beta-vae: Learning basic visual concepts with a constrained variational framework. *ICLR (Poster)*, 3, 2017.
- [30] Hyunjik Kim and Andriy Mnih. Disentangling by factorising. In *International conference on machine learning*, pages 2649–2658. PMLR, 2018.
- [31] Minyoung Kim, Yuting Wang, Pritish Sahu, and Vladimir Pavlovic. Relevance factor vae: Learning and identifying disentangled factors. *arXiv preprint arXiv:1902.01568*, 2019.
- [32] Raphael Suter, Djordje Miladinovic, Bernhard Schölkopf, and Stefan Bauer. Robustly disentangled causal mechanisms: Validating deep representations for interventional robustness. In *International Conference on Machine Learning*, pages 6056–6065. PMLR, 2019.
- [33] Abhishek Kumar, Prasanna Sattigeri, and Avinash Balakrishnan. Variational inference of disentangled latent concepts from unlabeled observations. *arXiv preprint arXiv:1711.00848*, 2017.
- [34] Francesco Locatello, Stefan Bauer, Mario Lucic, Gunnar Raetsch, Sylvain Gelly, Bernhard Schölkopf, and Olivier Bachem. Challenging common assumptions in the unsupervised learning of disentangled representations. In *international conference on machine learning*, pages 4114–4124. PMLR, 2019.
- [35] Karl Ridgeway and Michael C Mozer. Learning deep disentangled embeddings with the f-statistic loss. *Advances in neural information processing systems*, 31, 2018.
- [36] Cian Eastwood and Christopher KI Williams. A framework for the quantitative evaluation of disentangled representations. In *International conference on learning representations*, 2018.
- [37] Antonio Almodévar, Alfonso Ortega, Luis Vicente, Antonio Miguel, and Eduardo Lleida. Defining and measuring disentanglement for non-independent factors of variation. *arXiv preprint arXiv:2408.07016*, 2024.
- [38] Ryan Rifkin and Aldebaro Klautau. In defense of one-vs-all classification. *Journal of machine learning research*, 5(Jan):101–141, 2004.
- [39] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006. ISBN 978-0-387-31073-2.

- [40] Diederik P Kingma. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*, 2013.
- [41] Jake Snell, Kevin Swersky, and Richard Zemel. Prototypical networks for few-shot learning. *Advances in neural information processing systems*, 30, 2017.
- [42] Antonio Almodévar, Alfonso Ortega, Luis Vicente, Antonio Miguel, and Eduardo Lleida. Variational classifier for unsupervised anomalous sound detection under domain generalization. In *Proceedings of INTERSPEECH*, pages 2823–2827, 2023.
- [43] Alex Krizhevsky, Geoffrey Hinton, et al. Learning multiple layers of features from tiny images. 2009.
- [44] Muhammad Waleed Gondal, Manuel Wuthrich, Djordje Miladinovic, Francesco Locatello, Martin Breidt, Valentin Volchkov, Joel Akpo, Olivier Bachem, Bernhard Schölkopf, and Stefan Bauer. On the transfer of inductive bias from simulation to the real world: a new disentanglement dataset. *Advances in Neural Information Processing Systems*, 32, 2019.
- [45] Ziwei Liu, Ping Luo, Xiaogang Wang, and Xiaoou Tang. Deep learning face attributes in the wild. In *Proceedings of International Conference on Computer Vision (ICCV)*, December 2015.
- [46] Catherine Wah, Steve Branson, Peter Welinder, Pietro Perona, and Serge Belongie. The caltech-ucsd birds-200-2011 dataset. 2011.
- [47] Naftali Tishby and Noga Zaslavsky. Deep learning and the information bottleneck principle. In *2015 IEEE information theory workshop (itw)*, pages 1–5. Ieee, 2015.
- [48] Michael Tschannen, Josip Djolonga, Paul K Rubenstein, Sylvain Gelly, and Mario Lucic. On mutual information maximization for representation learning. *arXiv preprint arXiv:1907.13625*, 2019.
- [49] Martin Arjovsky, Léon Bottou, Ishaan Gulrajani, and David Lopez-Paz. Invariant risk minimization. *arXiv preprint arXiv:1907.02893*, 2019.
- [50] Nello Cristianini, John Shawe-Taylor, Andre Elisseeff, and Jaz Kandola. On kernel-target alignment. *Advances in neural information processing systems*, 14, 2001.
- [51] Corinna Cortes, Mehryar Mohri, and Afshin Rostamizadeh. Algorithms for learning kernels based on centered alignment. *The Journal of Machine Learning Research*, 13(1):795–828, 2012.
- [52] Simon Kornblith, Mohammad Norouzi, Honglak Lee, and Geoffrey Hinton. Similarity of neural network representations revisited. In *International conference on machine learning*, pages 3519–3529. PMLR, 2019.

A Complete Graphical Models

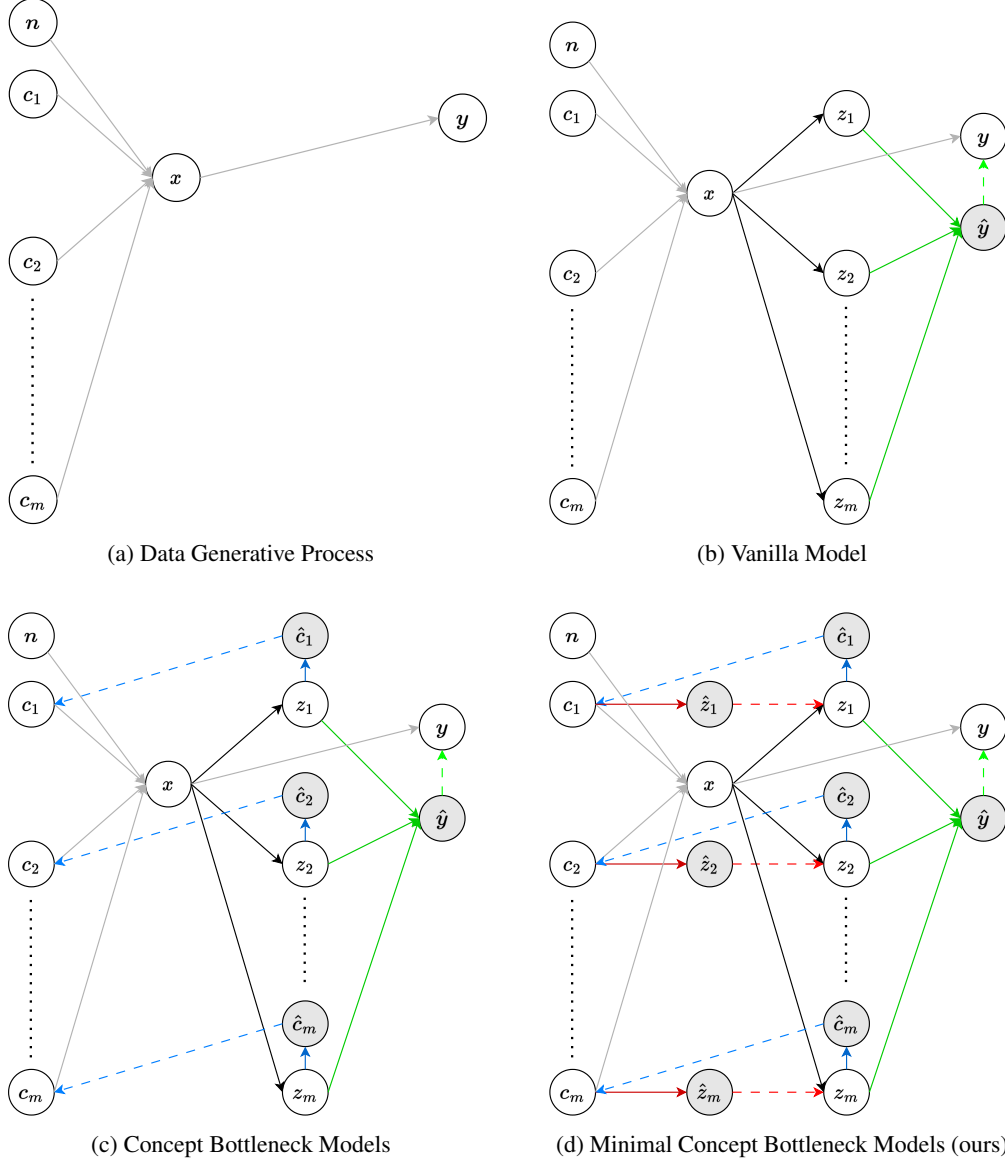


Figure 9: Graphical models of the different systems described. We consider m concepts and m -dimensional representations. Inputs $\mathbf{x}$ are defined by some concepts $\{c_j\}_{j=1}^m$ and nuisances $\mathbf{n}$; and targets $\mathbf{y}$ are defined by $\mathbf{x}$ (gray arrows). Vanilla models obtain the representations $\{z_j\}_{j=1}^m$ from $\mathbf{x}$ through the *encoder* $p_\theta(z|x)$ (black arrows) and solve the task $\hat{\mathbf{y}}$ sequentially through the *task head* $q_\phi(y|z)$ (green arrows). Concept Bottleneck Models try to make a prediction $\hat{c}_j$ of each concept c_j from one representation z_j through the *concept head* $q(\hat{c}_j|z_j)$ (blue arrows). Minimal CBMs (ours) try to make a prediction $\hat{z}_j$ of each representation z_j from one concept c_j through the *representation head* $q(\hat{z}_j|c_j)$ (red arrows).

B Details of Derivations

B.1 Proof of Equation 1

$$I(Z; Y) = \iint p(z, y) \log \frac{p(y|z)}{p(y)} dy dz \quad (7)$$

$$= \iiint p(x, y) p_\theta(z|x) \log \frac{p(y|z)}{p(y)} dx dy dz \quad (8)$$

$$= \iiint p(x, y) p_\theta(z|x) \log \frac{p(y|z)}{p(y)} \frac{q_\phi(\hat{y}|z)}{q_\phi(\hat{y}|z)} dx dy dz \quad (9)$$

$$= \mathbb{E}_{p(x, y)} [\mathbb{E}_{p_\theta(z|x)} [\log q_\phi(\hat{y}|z)]] + E_{p_\theta(z)} [D_{KL}(p(y|z) || q_\phi(\hat{y}|z))] + H(Y) \quad (10)$$

$$\geq \mathbb{E}_{p(x, y)} [\mathbb{E}_{p_\theta(z|x)} [\log q_\phi(\hat{y}|z)]] + H(Y) \quad (11)$$

Thus, since $H(Y)$ is independent of θ and ϕ , we have that:

$$\max_Z I(Z; Y) = \max_{\theta, \phi} \mathbb{E}_{p(x, y)} [\mathbb{E}_{p_\theta(z|x)} [\log q_\phi(\hat{y}|z)]] \quad (12)$$

B.2 Proof of Equation 2

$$I(Z_j; C_j) = \iint p(z_j, c_j) \log \frac{p(c_j|z_j)}{p(c_j)} dc_j dz_j \quad (13)$$

$$= \iiint p(x, c_j) p_\theta(z_j|x) \log \frac{p(c_j|z_j)}{p(c_j)} dx dc_j dz_j \quad (14)$$

$$= \iiint p(x, c_j) p_\theta(z_j|x) \log \frac{p(c_j|z_j)}{p(c_j)} \frac{q(\hat{c}_j|z_j)}{q(\hat{c}_j|z_j)} dx dc_j dz_j \quad (15)$$

$$\geq E_{p(x, c_j)} [E_{p_\theta(z_j|x)} [\log q(\hat{c}_j|z_j)]] + E_{p_\theta(z_j)} [D_{KL}(p(c_j|z_j) || q(\hat{c}_j|z_j))] + H(C_j) \quad (16)$$

$$\geq E_{p(x, c_j)} [E_{p_\theta(z_j|x)} [\log q(\hat{c}_j|z_j)]] + H(C_j) \quad (17)$$

Given the fact that $H(C_j)$ is constant, we have that:

$$\max_{Z_j} I(Z_j; C_j) = \max_{\theta} E_{p(x, c_j)} [E_{p_\theta(z_j|x)} [\log q(\hat{c}_j|z_j)]] \quad (18)$$

B.3 Proof of Equation 4

$$I(Z_j; X|C_j) = \iiint p(x, c_j) p_\theta(z_j|x) \log \frac{p(z_j|x)}{p(z_j|c_j)} dx dc_j dz_j \quad (19)$$

$$= \iiint p(x, c_j) p_\theta(z_j|x) \log \frac{p(z_j|x)}{p(z_j|c_j)} \frac{q(\hat{z}_j|c_j)}{q(\hat{z}_j|c_j)} dx dc_j dz_j \quad (20)$$

$$= \mathbb{E}_{p(x, c_j)} [D_{KL}(p_\theta(z_j|x) || q(\hat{z}_j|c_j))] - E_{p(c_j)} [D_{KL}(p(z_j|c_j) || q(\hat{z}_j|c_j))] \quad (21)$$

$$\leq \mathbb{E}_{p(x, c_j)} [D_{KL}(p_\theta(z_j|x) || q(\hat{z}_j|c_j))] \quad (22)$$

Thus, we have that:

$$\min_{Z_j} I(Z_j; X|C_j) = \min_{\theta} E_{p(x, c_j)} [D_{KL}(p_\theta(z_j|x) || q(\hat{z}_j|c_j))] \quad (23)$$

B.4 KL Divergence between two Gaussian Distributions

We are given the conditional distributions:

$$\begin{aligned} p_\theta(z_j|x) &= \mathcal{N}(f_\theta(x)_j, \sigma_x^2 I), \\ q_\phi(\hat{z}_j|c_j) &= \mathcal{N}(g_\phi^z(c_j), \sigma_z^2 I), \end{aligned}$$

and aim to minimize the expected KL divergence:

$$\min_{\theta, \phi} \mathbb{E}_{p(x, c_j)} [D_{\text{KL}}(p_\theta(z_j|x) \parallel q_\phi(\hat{z}_j|c_j))].$$

The KL divergence between two multivariate Gaussians with diagonal covariances is given by:

$$D_{\text{KL}}(\mathcal{N}(\mu_p, \Sigma_p) \parallel \mathcal{N}(\mu_q, \Sigma_q)) = \frac{1}{2} \left[\text{tr}(\Sigma_q^{-1} \Sigma_p) + (\mu_q - \mu_p)^\top \Sigma_q^{-1} (\mu_q - \mu_p) - d + \log \frac{\det \Sigma_q}{\det \Sigma_p} \right].$$

Applying this to our case:

- $\mu_p = f_\theta(x)_j, \mu_q = g_\phi^z(c_j),$
- $\Sigma_p = \sigma_x^2 I, \Sigma_q = \sigma_z^2 I,$
- d is the dimension of z_j .

Plugging in, we obtain:

$$D_{\text{KL}} = \frac{1}{2} \left[\frac{d\sigma_x^2}{\sigma_z^2} + \frac{1}{\sigma_z^2} \|f_\theta(x)_j - g_\phi^z(c_j)\|^2 - d + d \log \left(\frac{\sigma_z^2}{\sigma_x^2} \right) \right].$$

Note that all terms except the squared distance are constant with respect to θ and ϕ . Therefore:

$$\mathbb{E}_{p(x, c_j)} [D_{\text{KL}}(p_\theta(z_j|x) \parallel q_\phi(\hat{z}_j|c_j))] = \frac{1}{2\sigma_z^2} \mathbb{E}_{p(x, c_j)} [\|f_\theta(x)_j - g_\phi^z(c_j)\|^2] + \text{const.}$$

Conclusion: Minimizing the expected KL divergence

$$\min_{\theta, \phi} \mathbb{E}_{p(x, c_j)} [D_{\text{KL}}(p_\theta(z_j|x) \parallel q_\phi(\hat{z}_j|c_j))]$$

is equivalent (up to a scaling factor) to minimizing the expected mean squared error:

$$\min_{\theta, \phi} \mathbb{E}_{p(x, c_j)} [\|f_\theta(x)_j - g_\phi^z(c_j)\|^2].$$

C Training Algorithm of MCBMs

Algorithm 1 Training Algorithm for MCBMs

Input: Dataset $\mathcal{D} = \{\mathbf{x}^{(k)}, \mathbf{y}^{(k)}, \mathbf{c}^{(k)}\}_{k=1}^N$, latent norm λ , learning rate η , batch size B
Output: Parameters θ (encoder), ϕ (class-head, task-heads, representation-heads)

- 1: Initialize parameters θ , ϕ and representations heads:
- 2: **for all** $j = 1, \dots, n$ **do**
- 3: **if** c_j is binary **then**
- 4: $g_j^z \leftarrow \lambda$ if $c_j = 1$ else $-\lambda$
- 5: **else if** c_j is multiclass **then**
- 6: $g_j^z \leftarrow \lambda \cdot \text{one_hot}(c_j)$
- 7: **else**
- 8: $g_j^z \leftarrow \lambda \cdot c_j$
- 9: **end if**
- 10: **end for**
- 11: **while** not converged **do**
- 12: Sample a mini-batch $\{\mathbf{x}^{(k)}, \mathbf{y}^{(k)}, \mathbf{c}^{(k)}\}_{k=1}^B \sim \mathcal{D}$
- 13: **for all** $\mathbf{x}^{(k)}, \mathbf{y}^{(k)}, \mathbf{c}^{(k)}$ in batch **do**
- 14: Encode: Compute $\mu_\theta^{(k)} \leftarrow f_\theta(\mathbf{x}^{(k)})$
- 15: Sample noise $\epsilon \sim \mathcal{N}(0, I)$ ▷ Reparameterization trick with only one sample
- 16: Reparameterize: $\mathbf{z}^{(k)} \leftarrow \mu_\theta^{(k)} + \sigma_x \odot \epsilon$
- 17: Task prediction: $\hat{\mathbf{y}}^{(k)} \leftarrow g_\phi^y(\mathbf{z}^{(k)})$ ▷ Similar to VMs
- 18: Task loss: $\mathcal{L}_y^{(k)} \leftarrow \|\mathbf{y}^{(k)} - \hat{\mathbf{y}}^{(k)}\|^2$ if $\mathbf{y}$ is continuous else CE $(\mathbf{y}^{(k)}, \hat{\mathbf{y}}^{(k)})$
- 19: **for all** $j = 1, \dots, n$ **do**
- 20: Concept j prediction: $\hat{c}_j^{(k)} \leftarrow g_{\phi,j}^c(\mathbf{z}_j^{(k)})$ ▷ Similar to CBMs
- 21: Concept j loss: $\mathcal{L}_{c,j}^{(k)} \leftarrow \|c_j^{(k)} - \hat{c}_j^{(k)}\|^2$ if c_j is continuous else CE $(c_j^{(k)}, \hat{c}_j^{(k)})$
- 22: Representation j prediction: $\hat{z}_j^{(k)} \leftarrow g_j^z(c_j^{(k)})$ ▷ Novelty in MCBMs
- 23: Representation j loss: $\mathcal{L}_{z,j}^{(k)} \leftarrow \|z_j^{(k)} - \hat{z}_j^{(k)}\|^2$
- 24: **end for**
- 25: Total loss: $\mathcal{L}^{(k)} \leftarrow \mathcal{L}_y^{(k)} + \beta \sum_{j=1}^n \mathcal{L}_{c,j}^{(k)} + \gamma \sum_{j=1}^n \mathcal{L}_{z,j}^{(k)}$
- 26: **end for**
- 27: Update θ, ϕ using gradient descent:

$$\theta \leftarrow \theta - \eta \nabla_\theta \left(\frac{1}{B} \sum_{k=1}^B \mathcal{L}^{(k)} \right), \quad \phi \leftarrow \phi - \eta \nabla_\phi \left(\frac{1}{B} \sum_{k=1}^B \mathcal{L}^{(k)} \right)$$

28: **end while**

D Experiments Details

D.1 Hyperparameters for Section 5

Table 5: Hyperparameters for Section 5. For all datasets, the concept head g_ϕ^c is implemented as the identity function in CBMs, and as a multilayer perceptron (MLP) with three hidden layers in MCBMs. We emphasize that CBMs are, by design, restricted to use invertible g_ϕ^c to enable intervention procedures.

	CIFAR-10	MPI3D	Shapes3D	CelebA	CUB
f_θ architecture	ResNet20	ResNet20	ResNet20	ResNet18	InceptionV3
f_θ pretraining	None	None	None	ImageNet	ImageNet
g_ϕ^y hidden layers	64	64	64	64	256
low γ	1	1	1	0.05	0.1
medium γ	5	3	3	0.1	0.3
high γ	10	5	5	0.3	0.5
number of epochs	50	50	50	120	250
batch size	128	128	128	128	128
optimizer	SGD	SGD	SGD	SGD	SGD
learning rate	6×10^{-3}	6×10^{-3}	6×10^{-3}	2×10^{-2}	2×10^{-2}
momentum	0.9	0.9	0.9	0.9	0.9
weight decay	4×10^{-5}	4×10^{-5}	4×10^{-5}	4×10^{-5}	4×10^{-5}
scheduler	Step	Step	Step	Step	Step
step size (epochs)	20	20	20	50	100
scheduler γ	0.1	0.1	0.1	0.1	0.1

D.2 Datasets

CIFAR-10 The CIFAR-10 dataset consists of 60,000 color images of size 32×32, evenly distributed across 10 object classes, with 50,000 training and 10,000 test samples. Among these, 6 classes correspond to animals and 4 to vehicles. We define the target task y as a binary classification between *animals* and *vehicles*. The subset n_y includes one vehicle and one animal class—specifically, *airplane* and *bird*—while c comprises the remaining 8 object classes.

MPI3D This is a synthetic dataset with controlled variation across seven generative factors: *object shape*, *object color*, *object size*, *camera height*, *background color*, *horizontal axis*, and *vertical axis*. In our setup, y corresponds to the *object shape*, n_y to the *horizontal axis*, $n_{\bar{y}}$ to the *vertical axis*, and c to the remaining generative factors. To ensure consistency in the mapping between concepts and the target, we filter the dataset such that any combination of elements in $\{c, n_y\}$ corresponds to a unique value of y . All invalid combinations are removed accordingly.

Shapes3D This synthetic dataset consists of 3D-rendered objects placed in a room, with variation across six known generative factors: *floor color*, *wall color*, *object color*, *scale*, *shape*, and *orientation*. In our setup, y corresponds to the *shape*, n_y includes *floor color* and *wall color*, $n_{\bar{y}}$ corresponds to *orientation*, and c comprises the remaining factors. We follow the same filtering strategy as in MPI3D to construct this configuration: we retain only those samples for which each combination of $\{c, n_y\}$ uniquely determines y , removing all invalid configurations.

CelebA : The CelebA dataset consists of over 200,000 celebrity face images, each annotated with 40 binary facial attributes. In our setup, the task variable y corresponds to binary gender classification. For each run, eight attributes are randomly selected as the concept set c , while the remaining attributes are treated as nuisances n_y . Since most attributes exhibit some degree of correlation with gender, we set $n_{\bar{y}}$ to the empty set.

CUB : The Caltech-UCSD Birds (CUB) dataset contains 11,788 images of 200 bird species, annotated with part locations, bounding boxes, and 312 binary attributes. Following the approach of [9], we retain only the attributes that are present in at least 10 species (based on majority voting), resulting in a filtered set of 112 attributes. These attributes are grouped into 27 semantic clusters, where each group is defined by a common prefix in the attribute names. In our setup, the task variable y is to classify the bird species. The concept set c consists of the attributes belonging to 7 randomly selected groups (per run), while the nuisance set n_y includes the attributes from the remaining 20 groups. Since most attributes exhibit some correlation with the classification task, we set $n_{\bar{y}}$ to the empty set.